

Отчёт по лабораторной работе №6

Управление процессами

Лабси Мохаммед

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	6.4. Управление процессами и заданиями	6
2.2	6.5. Самостоятельная работа	12
2.3	Контрольные вопросы	19
3	Заключение	23

Список иллюстраций

2.1	Запуск, приостановка, продолжение и завершение фоновых заданий	8
2.2	Поиск работающего процесса dd с помощью top	9
2.3	Состояние системы после завершения процесса dd	10
2.4	Запуск процессов dd, их поиск и изменение приоритета	11
2.5	Просмотр иерархии процессов и завершение корневой оболочки	12
2.6	Изменение значения nice и завершение процессов dd	13
2.7	Запуск программы yes в фоновом и переднем режимах	14
2.8	Управление заданиями yes и запуск процесса через nohup	15
2.9	Проверка процессов yes с помощью top	16
2.10	Завершение процессов yes по PID, номеру задания, сигналом SIGHUP и командой killall	18
2.11	Сравнение приоритетов процессов yes и их выравнивание с помощью renice	19

Список таблиц

1 Цель работы

Получить навыки управления процессами операционной системы.

2 Выполнение лабораторной работы

2.1 6.4. Управление процессами и заданиями

2.1.1 6.4.1. Управление заданиями

Получаю полномочия администратора:

```
su -
```

После успешной аутентификации запускаю два процесса в фоновом режиме:

```
sleep 3600 &
```

```
dd if=/dev/zero of=/dev/null &
```

Знак & в конце команды приводит к запуску процесса в фоновом режиме, поэтому оболочка сразу возвращает приглашение для ввода следующей команды.

Затем запускаю ещё один процесс:

```
sleep 7200
```

Последняя команда запущена без символа &, поэтому процесс выполняется на переднем плане и блокирует ввод команд в оболочке. Для временной приостановки процесса использую сочетание клавиш Ctrl+Z.

После этого проверяю список заданий:

```
jobs
```

В списке отображаются три задания. Процессы `sleep 3600` и `dd if=/dev/zero of=/dev/null` имеют состояние Running, а процесс `sleep 7200`, остановленный

сочетанием `Ctrl+Z`, имеет состояние `Stopped`.

Для продолжения выполнения третьего задания в фоновом режиме выполняю:

```
bg 3
```

Повторно проверяю состояние заданий:

```
jobs
```

После выполнения команды `bg 3` процесс `sleep 7200` переходит из состояния `Stopped` в состояние `Running` и продолжает работу в фоне.

Далее переношу первое задание на передний план:

```
fg 1
```

После перехода процесса `sleep 3600` на передний план завершаю его сочетанием клавиш `Ctrl+C`.

Аналогично перевожу на передний план второе задание:

```
fg 2
```

Процесс `dd` начинает выполняться на переднем плане. Завершаю его сочетанием `Ctrl+C`. После завершения `dd` выводится статистика обработанных данных.

Затем переношу на передний план третье задание:

```
fg 3
```

Процесс `sleep 7200` также завершаю сочетанием клавиш `Ctrl+C`.

В результате все три созданных задания прекращают работу (см. рис. Рисунок 2.1).

```

mohammedlabsi@mohammedlabsi:~$ su
Password:
root@mohammedlabsi:/home/mohammedlabsi#
root@mohammedlabsi:/home/mohammedlabsi# sleep 3600 &
[1] 3828
root@mohammedlabsi:/home/mohammedlabsi# dd if=/dev/zero of=/dev/null &
[2] 3870
root@mohammedlabsi:/home/mohammedlabsi# sleep 7200
^Z
[3]+  Stopped                  sleep 7200
root@mohammedlabsi:/home/mohammedlabsi# jobs
[1]  Running                  sleep 3600 &
[2]-  Running                  dd if=/dev/zero of=/dev/null &
[3]+  Stopped                  sleep 7200
root@mohammedlabsi:/home/mohammedlabsi# bg 3
[3]+ sleep 7200 &
root@mohammedlabsi:/home/mohammedlabsi# jobs
[1]  Running                  sleep 3600 &
[2]-  Running                  dd if=/dev/zero of=/dev/null &
[3]+  Running                  sleep 7200 &
root@mohammedlabsi:/home/mohammedlabsi# fg 1
sleep 3600
^C
root@mohammedlabsi:/home/mohammedlabsi# fg 2
dd if=/dev/zero of=/dev/null
^C435197289+0 records in
435197289+0 records out
222821011968 bytes (223 GB, 208 GiB) copied, 83.8814 s, 2.7 GB/s

root@mohammedlabsi:/home/mohammedlabsi# fg 3
sleep 7200
^C
root@mohammedlabsi:/home/mohammedlabsi# █

```

Рисунок 2.1: Запуск, приостановка, продолжение и завершение фоновых заданий

Далее открываю второй терминал и под учётной записью обычного пользователя запускаю процесс dd в фоновом режиме:

```
dd if=/dev/zero of=/dev/null &
```

После запуска фонового процесса завершаю оболочку второго терминала командой:

```
exit
```

В другом терминале запускаю утилиту мониторинга процессов:

```
top
```

В списке процессов присутствует процесс dd с PID 4179, запущенный пользователем. В момент проверки он использует около 99,3 % процессорного времени, что подтверждает его активную работу после закрытия терминала, из которого он был запущен (см. рис. Рисунок 2.2).

```

top - 13:24:08 up 5 min, 2 users, load average: 0.67, 0.52, 0.24
Tasks: 286 total, 2 running, 284 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.5 us, 8.4 sy, 0.0 ni, 86.7 id, 0.0 wa, 0.4 ht, 0.0 si, 0.0 st
MiB Mem : 3988.7 total, 1621.4 free, 1489.6 used, 1045.0 buff/cache
MiB Swap: 4040.0 total, 4040.0 free, 0.0 used, 2419.1 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
4179	mohamme+	20	0	226864	1872	1748	R	99.3	0.0	0:19.43	dd
2505	mohamme+	20	0	4925932	309676	121688	S	2.3	7.7	0:02.43	gnome-shell
3620	mohamme+	20	0	3040524	242944	101848	S	1.7	6.1	0:01.26	ptyxis
76	root	20	0	0	0	0	I	0.3	0.0	0:00.06	kworker/u19:2-events_unbound
1	root	20	0	44676	36548	10488	S	0.0	0.9	0:01.39	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
9	root	20	0	0	0	0	I	0.0	0.0	0:00.02	kworker/0:0-events_freezable_pwr_efficient
10	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/0:1-cgroup_destroy
11	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-events_highpri
12	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/u16:0-lpvs_addrconf
13	root	20	0	0	0	0	I	0.0	0.0	0:00.02	kworker/u16:1-netns
14	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
15	root	20	0	0	0	0	S	0.0	0.0	0:00.01	ksoftirqd/0
16	root	20	0	0	0	0	I	0.0	0.0	0:00.09	rcu_preempt
17	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_par_gp_kthread_worker/0
18	root	20	0	0	0	0	S	0.0	0.0	0:00.03	rcu_exp_gp_kthread_worker
19	root	rt	0	0	0	0	S	0.0	0.0	0:00.02	migration/0
20	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/0
21	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
22	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
23	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/1
24	root	rt	0	0	0	0	S	0.0	0.0	0:00.18	migration/1
25	root	20	0	0	0	0	S	0.0	0.0	0:00.02	ksoftirqd/1
26	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/1:0-events

Рисунок 2.2: Поиск работающего процесса dd с помощью top

Для завершения процесса повторно использую:

top

В интерфейсе top нажимаю клавишу k, указываю PID процесса 4179 и подтверждаю отправку сигнала завершения.

После этого процесс dd исчезает из списка наиболее активных процессов. В верхней части списка больше нет процесса с PID 4179, использовавшего почти всё доступное процессорное время (см. рис. Рисунок 2.3).

```

top - 13:24:22 up 5 min, 2 users, load average: 0.75, 0.54, 0.25
Tasks: 285 total, 4 running, 281 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.2 us, 7.9 sy, 0.0 ni, 87.4 id, 0.0 wa, 0.3 hi, 0.1 si, 0.0 st
MiB Mem : 3988.7 total, 1630.5 free, 1480.2 used, 1045.4 buff/cache
MiB Swap: 4040.0 total, 4040.0 free, 0.0 used, 2428.4 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3620	mohamme+	20	0	3040524	245896	101848	S	1.9	6.1	0:01.48	ptxyls
2505	mohamme+	20	0	4929724	309760	121688	S	1.7	7.7	0:02.76	gnome-shell
76	root	20	0	0	0	0	I	0.2	0.0	0:00.12	kworker/u19:2-events_unbound
1436	apache	20	0	2157808	7748	4188	S	0.2	0.2	0:00.07	htpdp
1	root	20	0	44676	36548	10488	S	0.0	0.9	0:01.40	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
9	root	20	0	0	0	0	I	0.0	0.0	0:00.03	kworker/0:0-rcu_gp
10	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/0:1-cgroup_destroy
11	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-events_highpri
12	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/u16:0-lpv6_addrconf
13	root	20	0	0	0	0	I	0.0	0.0	0:00.02	kworker/u16:1-netns
14	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq
15	root	20	0	0	0	0	S	0.0	0.0	0:00.01	ksoftirqd/0
16	root	20	0	0	0	0	R	0.0	0.0	0:00.09	rcu_preempt
17	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_exp_par_gp_kthread_worker/0
18	root	20	0	0	0	0	S	0.0	0.0	0:00.03	rcu_exp_gp_kthread_worker
19	root	rt	0	0	0	0	S	0.0	0.0	0:00.02	migration/0
20	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/0
21	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
22	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
23	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inject/1
24	root	rt	0	0	0	0	S	0.0	0.0	0:00.18	migration/1
25	root	20	0	0	0	0	S	0.0	0.0	0:00.02	ksoftirqd/1
26	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/1:0-events

Рисунок 2.3: Состояние системы после завершения процесса dd

2.1.2 6.4.2. Управление процессами

Получаю полномочия администратора:

su -

Запускаю три экземпляра программы dd в фоновом режиме:

dd if=/dev/zero of=/dev/null &

dd if=/dev/zero of=/dev/null &

dd if=/dev/zero of=/dev/null &

Созданным процессам присваиваются PID 4460, 4462 и 4464.

Для поиска процессов выполняю:

ps aux | grep dd

Команда ps aux выводит список процессов системы, а grep dd оставляет строки, содержащие последовательность dd. Помимо системных процессов, в названиях которых встречается эта последовательность, в конце списка отображаются три запущенных процесса:

dd if=/dev/zero of=/dev/null

Для процесса с PID 4460 изменяю значение nice:

```
renice -n 5 4460
```

Утилита сообщает:

```
4460 (process ID) old priority 0, new priority 5
```

Следовательно, значение nice процесса было изменено с 0 на 5. Увеличение значения nice уменьшает приоритет процесса при распределении процессорного времени (см. рис. Рисунок 2.4).

```
root@mohammedlabsi:/home/mohammedlabsi# dd if=/dev/zero of=/dev/null &
[1] 4460
root@mohammedlabsi:/home/mohammedlabsi# dd if=/dev/zero of=/dev/null &
[2] 4462
root@mohammedlabsi:/home/mohammedlabsi# dd if=/dev/zero of=/dev/null &
[3] 4464
root@mohammedlabsi:/home/mohammedlabsi# ps aux | grep dd
root      2  0.0  0.0      0  0 ?        S   13:19   0:00 [kthreadd]
root     12  0.0  0.0      0  0 ?        I   13:19   0:00 [kworker/u16:0-ipv6_addrconf]
root     105  0.0  0.0      0  0 ?        I<  13:19   0:00 [kworker/R-ipv6_addrconf]
root     1628 0.0  0.0  512976 3304 ?        Sl  13:19   0:00 /usr/sbin/VBoxService --pidfile /var/run/vboxadd-service.sh
mohamme+ 2906  0.0  0.6 1037540 25832 ?        Ssl 13:19   0:00 /usr/libexec/evolution-addressbook-factory
root     4460 99.0  0.0 226864 1892 pts/0    R   13:25   0:20 dd if=/dev/zero of=/dev/null
root     4462 98.6  0.0 226864 1892 pts/0    R   13:25   0:19 dd if=/dev/zero of=/dev/null
root     4464 98.8  0.0 226864 1976 pts/0    R   13:25   0:18 dd if=/dev/zero of=/dev/null
root     4511  0.0  0.0 227704 2176 pts/0    S+  13:25   0:00 grep --color=auto dd
root@mohammedlabsi:/home/mohammedlabsi# renice -n 5 4460
4460 (process ID) old priority 0, new priority 5
root@mohammedlabsi:/home/mohammedlabsi# ps fax | grep -B5 dd
PID TTY          STAT       TIME COMMAND
PID TTY          STAT       TIME COMMAND
```

Рисунок 2.4: Запуск процессов dd, их поиск и изменение приоритета

Для просмотра иерархии процессов выполняю:

```
ps fax | grep -B5 dd
```

Параметр f команды ps отображает процессы в виде дерева, а параметр -B5 программы gtop дополнительно показывает пять строк, расположенных перед найденными совпадениями.

В полученном дереве видно, что три процесса dd с PID 4460, 4462 и 4464 были запущены из оболочки bash с PID 3769. Выше в дереве также отображается процесс su с PID 3721 и исходная пользовательская оболочка.

Для завершения корневой оболочки отправляю процессу с PID 3769 сигнал SIGKILL:

```
kill -9 3769
```

После выполнения команды корневая оболочка завершается. Команда демонстрирует возможность управления процессами через их PID и работу с иерархией родительских и дочерних процессов (см. рис. Рисунок 2.5).

```

3620 ?      Ssl  0:02  \_ /usr/bin/ptyxis --gapapplication-service
3628 ?      Ssl  0:00  |  \_ /usr/libexec/ptyxis-agent --socket-fd=3 --rlimit-nofile=1024
3691 pts/0  Ss   0:00  |  \_ /usr/bin/bash
3721 pts/0  S    0:00  |  |  \_ su
3769 pts/0  S    0:00  |  |  \_ bash
4460 pts/0  RN   0:56  |  |  \_ dd if=/dev/zero of=/dev/null
4462 pts/0  R    0:55  |  |  \_ dd if=/dev/zero of=/dev/null
4464 pts/0  R    0:55  |  |  \_ dd if=/dev/zero of=/dev/null
4588 pts/0  R+   0:00  |  |  \_ ps fax
4589 pts/0  S+   0:00  |  |  \_ grep --color=auto -B5 dd
root@mohammedlabsi:/home/mohammedlabsi# kill -9 3769

```

Рисунок 2.5: Просмотр иерархии процессов и завершение корневой оболочки

2.2 6.5. Самостоятельная работа

2.2.1 6.5.1. Задание 1

Получаю полномочия администратора:

```
su -
```

Запускаю три процесса dd в фоновом режиме:

```
dd if=/dev/zero of=/dev/null &
```

```
dd if=/dev/zero of=/dev/null &
```

```
dd if=/dev/zero of=/dev/null &
```

Процессам присваиваются PID 4853, 4855 и 4857.

Для первого процесса изменяю значение nice:

```
renice -n 5 4853
```

Получаю результат:

```
4853 (process ID) old priority 0, new priority 5
```

После этого ещё раз изменяю значение nice того же процесса:

```
renice -n 15 4853
```

Система сообщает:

```
4853 (process ID) old priority 5, new priority 15
```

На выполненном скриншоте использованы положительные значения 5 и 15.

При увеличении nice с 5 до 15 процесс получает меньший приоритет при планировании процессорного времени.

По условию задания требовалось использовать значения -5 и -15. Для выполнения именно этого условия команды должны иметь вид:

```
renice -n -5 4853  
renice -n -15 4853
```

При переходе от -5 к -15 приоритет процесса, напротив, увеличивается: чем меньше значение nice, тем выше предпочтение процесса при распределении процессорного времени. Установка отрицательных значений nice требует соответствующих административных полномочий.

Для одновременного завершения всех запущенных процессов dd выполняю:

```
killall dd
```

После выполнения команды оболочка сообщает о завершении заданий [1], [2] и [3] (см. рис. Рисунок 2.6).

```
mohammedlabsi@mohammedlabsi:~$ su  
Password:  
root@mohammedlabsi:/home/mohammedlabsi#  
root@mohammedlabsi:/home/mohammedlabsi# dd if=/dev/zero of=/dev/null &  
[1] 4853  
root@mohammedlabsi:/home/mohammedlabsi# dd if=/dev/zero of=/dev/null &  
[2] 4855  
root@mohammedlabsi:/home/mohammedlabsi# dd if=/dev/zero of=/dev/null &  
[3] 4857  
root@mohammedlabsi:/home/mohammedlabsi# renice -n 5 4853  
4853 (process ID) old priority 0, new priority 5  
root@mohammedlabsi:/home/mohammedlabsi# renice -n 15 4853  
4853 (process ID) old priority 5, new priority 15  
root@mohammedlabsi:/home/mohammedlabsi# killall dd  
[1] Terminated dd if=/dev/zero of=/dev/null  
[2]- Terminated dd if=/dev/zero of=/dev/null  
[3]+ Terminated dd if=/dev/zero of=/dev/null  
root@mohammedlabsi:/home/mohammedlabsi# █
```

Рисунок 2.6: Изменение значения nice и завершение процессов dd

2.2.2 6.5.2. Задание 2

Запускаю программу yes в фоновом режиме, перенаправляя стандартный поток вывода в /dev/null:

```
yes > /dev/null &
```

Процесс запускается как фоновое задание [1] с PID 5265. Перенаправление в /dev/null необходимо, поскольку программа yes непрерывно выводит строки

и без перенаправления быстро заполняет терминал.

Затем запускаю аналогичный процесс на переднем плане:

```
yes > /dev/null
```

Приостанавливаю его сочетанием клавиш Ctrl+Z. оболочка сообщает, что задание [2] получило состояние Stopped.

Повторно запускаю:

```
yes > /dev/null
```

и завершаю процесс сочетанием клавиш Ctrl+C.

При запуске yes без подавления потока вывода используется команда:

```
yes
```

В этом случае программа непрерывно выводит символ y в терминал. Её также можно приостановить сочетанием Ctrl+Z, а после повторного запуска завершить сочетанием Ctrl+C (см. рис. Рисунок 2.7).

```
root@mohammedlabsi:/home/mohammedlabsi#  
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null &  
[1] 5265  
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null  
^Z  
[2]+  Stopped                  yes > /dev/null  
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null  
^C  
root@mohammedlabsi:/home/mohammedlabsi# █
```

Рисунок 2.7: Запуск программы yes в фоновом и переднем режимах

Проверяю состояние заданий:

```
jobs
```

В списке видно, что задание [1] находится в состоянии Running, а задание [2] — в состоянии Stopped.

Перевожу первое фоновое задание на передний план:

```
fg 1
```

После появления процесса на переднем плане завершаю его сочетанием клавиш Ctrl+C.

Затем возобновляю выполнение второго остановленного задания в фоне:

```
bg 2
```

Проверяю состояние:

```
jobs
```

Теперь задание [2] имеет состояние Running и выполняется в фоновом режиме.

Для запуска процесса, который должен быть защищён от сигнала SIGHUP при завершении терминального сеанса, использую nohup:

```
nohup yes > /dev/null &
```

Создаётся задание [3] с PID 5450. Утилита nohup сообщает об игнорировании стандартного ввода и перенаправлении стандартного потока ошибок. Процесс получает возможность продолжить работу после закрытия терминальной сессии (см. рис. Рисунок 2.8).

```
root@mohammedlabsi:/home/mohammedlabsi# jobs
[1]-  Running                  yes > /dev/null &
[2]+  Stopped                  yes > /dev/null
root@mohammedlabsi:/home/mohammedlabsi# fg 1
yes > /dev/null
^C
root@mohammedlabsi:/home/mohammedlabsi# bg 2
[2]+  yes > /dev/null &
root@mohammedlabsi:/home/mohammedlabsi# jobs
[2]+  Running                  yes > /dev/null &
root@mohammedlabsi:/home/mohammedlabsi# nohup yes > /dev/null &
[3] 5450
nohup: ignoring input and redirecting stderr to stdout
root@mohammedlabsi:/home/mohammedlabsi#
root@mohammedlabsi:/home/mohammedlabsi# █
```

Рисунок 2.8: Управление заданиями yes и запуск процесса через nohup

После закрытия окна терминала и повторного запуска консоли проверяю работающие процессы при помощи:

```
top
```

В верхней части списка отображаются процессы yes. В частности, присутствует процесс с PID 5450, ранее запущенный через nohup. Он продолжает выполняться и активно использует процессорное время.

Также в списке присутствует процесс yes с PID 5287. Оба процесса в момент наблюдения используют приблизительно по 90,9 % процессорного времени (см. рис. Рисунок 2.9).

```

top - 13:33:02 up 14 min, 3 users, load average: 1.52, 1.48, 0.94
Tasks: 279 total, 4 running, 275 sleeping, 0 stopped, 0 zombie
%Cpu(s): 15.2 us, 24.2 sy, 0.0 ni, 60.6 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3988.7 total, 1837.0 free, 1273.3 used, 1046.0 buff/cache
MiB Swap: 4040.0 total, 4040.0 free, 0.0 used, 2635.4 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 5287 root        20   0 226836 1920 1800 R  90.9   0.0   1:07.05 yes
 5450 root        20   0 226836 1936 1816 R  90.9   0.0   0:40.64 yes
    1 root         0   0  44676 36548 10488 S   0.0   0.9   0:01.84 systemd
    2 root         0   0      0      0      0 S   0.0   0.0   0:00.00 kthreadd
    3 root         0   0      0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root        -20   0      0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
    6 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
    7 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
    8 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/R-nets
    9 root         0   0      0      0      0 I   0.0   0.0   0:00.05 kworker/0:0-ata_sff
   11 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
   12 root         0   0      0      0      0 I   0.0   0.0   0:00.00 kworker/u16:0-lpv6_addrconf
   13 root         0   0      0      0      0 I   0.0   0.0   0:00.06 kworker/u16:1-nets
   14 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
   15 root         0   0      0      0      0 S   0.0   0.0   0:00.02 ksoftirqd/0
   16 root         0   0      0      0      0 I   0.0   0.0   0:00.17 rcu_preempt
   17 root         0   0      0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
   18 root         0   0      0      0      0 S   0.0   0.0   0:00.06 rcu_exp_gp_kthread_worker
   19 root         0   0      0      0      0 S   0.0   0.0   0:00.02 migration/0
   20 root        -51   0      0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
   21 root         0   0      0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
   22 root         0   0      0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
   23 root        -51   0      0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
   24 root         0   0      0      0      0 S   0.0   0.0   0:00.18 migration/1
   25 root         0   0      0      0      0 S   0.0   0.0   0:00.02 ksoftirqd/1
   27 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
   28 root         0   0      0      0      0 S   0.0   0.0   0:00.00 cpuhp/2
   29 root        -51   0      0      0      0 S   0.0   0.0   0:00.00 idle_inject/2
   30 root         0   0      0      0      0 S   0.0   0.0   0:00.17 migration/2
   31 root         0   0      0      0      0 S   0.0   0.0   0:00.01 ksoftirqd/2
   33 root         0 -20   0      0      0 I   0.0   0.0   0:00.00 kworker/2:0H-events_highpri

```

Рисунок 2.9: Проверка процессов yes с помощью top

Далее запускаю ещё три процесса yes в фоновом режиме с подавлением вывода:

```
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
```

Им присваиваются PID 5757, 5762 и 5769.

Первый процесс завершаю по его PID:

```
kill 5757
```

Оболочка сообщает о завершении задания [1].

Для работы со вторым процессом использую идентификатор задания:

```
fg 2
```

После перевода задания [2] на передний план завершаю его сочетанием клавиш Ctrl+C.

Далее проверяю действие сигнала SIGHUP, номер которого равен 1. Отправляю его обычному процессу:

```
kill -1 5287
```

После этого отправляю SIGHUP процессу с PID 5450, который был запущен через nohup:

```
kill -1 5450
```

Процесс, запущенный через nohup, предназначен для игнорирования сигнала SIGHUP, поэтому он должен продолжить выполнение после получения этого сигнала.

Также отправляю сигнал SIGHUP обычному процессу с PID 5769:

```
kill -1 5769
```

Для него оболочка выводит сообщение:

```
Hangup
```

Это показывает, что обычный процесс завершился при получении SIGHUP, в отличие от процесса, защищённого с помощью nohup.

После этого запускаю ещё несколько экземпляров yes:

```
yes > /dev/null &
```

```
yes > /dev/null &
```

Создаются процессы с PID 5984 и 5986.

Для их одновременного завершения использую команду:

```
killall yes
```

Команда завершает процессы программы yes, а оболочка сообщает о завершении фоновых заданий (см. рис. Рисунок 2.10).

```

root@mohammedlabsi:/home/mohammedlabsi#
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null &
[1] 5757
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null &
[2] 5762
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null &
[3] 5769
root@mohammedlabsi:/home/mohammedlabsi# kill 5757
[1] Terminated yes > /dev/null
root@mohammedlabsi:/home/mohammedlabsi# fg 2
yes > /dev/null
^C
root@mohammedlabsi:/home/mohammedlabsi# kill -1 5287
root@mohammedlabsi:/home/mohammedlabsi# kill -1 5450
root@mohammedlabsi:/home/mohammedlabsi# kill -1 5769
[3]+ Hangup yes > /dev/null
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null &
[1] 5984
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null &
[2] 5986
root@mohammedlabsi:/home/mohammedlabsi# killall yes
[1]- Terminated yes > /dev/null
[2]+ Terminated yes > /dev/null
root@mohammedlabsi:/home/mohammedlabsi# █

```

Рисунок 2.10: Завершение процессов yes по PID, номеру задания, сигналом SIGHUP и командой killall

Для сравнения приоритетов запускаю обычный процесс yes:

```
yes > /dev/null &
```

Он получает PID 6132.

Затем с помощью утилиты nice запускаю второй процесс с увеличенным на 5 значением nice:

```
nice -n 5 yes > /dev/null &
```

Второй процесс получает PID 6165.

Для сравнения параметров процессов выполняю:

```
ps -l | grep yes
```

Для процесса с PID 6132 отображается значение NI, равное 0, а значение приоритета PRI составляет 80.

Для процесса с PID 6165, запущенного через nice -n 5, значение NI равно 5, а PRI — 85.

Положительное увеличение nice уменьшает фактический приоритет процесса. Поэтому процесс с NI=5 имеет меньшее предпочтение при распределении процессорного времени, чем процесс с NI=0.

Для выравнивания значений nice изменяю параметры первого процесса:

```
renice -n 5 6132
```

Система подтверждает изменение:

6132 (process ID) old priority 0, new priority 5

Повторно выполняю:

```
ps -l | grep yes
```

Теперь оба процесса имеют одинаковое значение NI=5 и одинаковое значение PRI=85. Следовательно, после выполнения `renice` их параметры приоритета были выровнены (см. рис. Рисунок 2.11).

```
root@mohammedlabsi:/home/mohammedlabsi#  
root@mohammedlabsi:/home/mohammedlabsi# yes > /dev/null &  
[1] 6132  
root@mohammedlabsi:/home/mohammedlabsi# nice -n 5 yes > /dev/null &  
[2] 6165  
root@mohammedlabsi:/home/mohammedlabsi# ps -l | grep yes  
4 R    0    6132    5629 99  80    0 - 56709 -      pts/1    00:00:19 yes  
4 R    0    6165    5629 99  85    5 - 56709 -      pts/1    00:00:05 yes  
root@mohammedlabsi:/home/mohammedlabsi# renice -n 5 6132  
6132 (process ID) old priority 0, new priority 5  
root@mohammedlabsi:/home/mohammedlabsi# ps -l | grep yes  
4 R    0    6132    5629 99  85    5 - 56709 -      pts/1    00:00:49 yes  
4 R    0    6165    5629 99  85    5 - 56709 -      pts/1    00:00:35 yes  
root@mohammedlabsi:/home/mohammedlabsi#
```

Рисунок 2.11: Сравнение приоритетов процессов `yes` и их выравнивание с помощью `renice`

2.3 Контрольные вопросы

1. Какая команда даёт обзор всех текущих заданий оболочки?

Для просмотра всех заданий, запущенных из текущей оболочки, используется команда **jobs**.

Она показывает номера заданий, их состояние (**Running, Stopped, Done**) и команды, соответствующие этим заданиям.

Например: **jobs**.

2. Как остановить текущее задание оболочки, чтобы продолжить его выполнение в фоновом режиме?

Для временной приостановки задания, выполняющегося на переднем плане, используется сочетание клавиш **Ctrl+Z**. После этого процесс получает состояние **Stopped**.

Для продолжения его выполнения в фоновом режиме используется команда **bg**.

Например: **bg 1**, где **1** — номер задания. Если необходимо продолжить последнее приостановленное задание, обычно достаточно выполнить **bg** без указания номера.

3. **Какую комбинацию клавиш можно использовать для отмены текущего задания оболочки?**

Для завершения текущего задания, выполняющегося на переднем плане, используется сочетание клавиш **Ctrl+C**.

При этом процессу отправляется сигнал **SIGINT**, который обычно приводит к завершению его работы.

4. **Необходимо отменить одно из начатых заданий. Доступ к оболочке, в которой в данный момент работает пользователь, невозможен. Что можно сделать, чтобы отменить задание?**

Можно открыть другой терминал, определить идентификатор процесса (**PID**) с помощью команд **ps** или **top**, после чего завершить процесс командой **kill**.

Например, сначала можно найти процесс: **ps aux | grep имя_процесса**, а затем выполнить **kill PID**.

Если процесс не реагирует на обычный сигнал завершения, можно использовать **kill -9 PID**, что отправляет сигнал **SIGKILL** и принудительно завершает процесс.

5. **Какая команда используется для отображения отношений между родительскими и дочерними процессами?**

Для отображения процессов в виде иерархического дерева можно использовать команду **ps fax**.

Она показывает отношения между родительскими и дочерними про-

цессами. Например, в лабораторной работе использовалась команда **ps fax | grep -B5 dd**, позволяющая дополнительно найти процессы **dd** и увидеть несколько предшествующих строк дерева процессов.

Для отдельного отображения дерева процессов также существует команда **pstree**.

6. **Какая команда позволит изменить приоритет процесса с идентификатором 1234 на более высокий?**

Для изменения приоритета уже запущенного процесса используется команда **renice**. Чтобы увеличить приоритет процесса, необходимо уменьшить его значение **nice**.

Например: **renice -n -5 1234**.

В данном случае процессу с PID **1234** устанавливается значение **nice** = **-5**, поэтому его приоритет по сравнению с обычным процессом с **nice** = **0** повышается. Установка отрицательных значений **nice** обычно требует полномочий администратора.

7. **В системе в настоящее время запущено 20 процессов dd. Как проще всего остановить их все сразу?**

Для одновременного завершения всех процессов с определённым именем удобно использовать команду **killall**.

В данном случае необходимо выполнить: **killall dd**.

Команда отправит сигнал завершения всем доступным процессам с именем **dd**.

8. **Какая команда позволяет остановить команду с именем mycommand?**

Для завершения процесса по его имени можно использовать команду **killall mycommand**.

Она отправит сигнал завершения процессам, имя которых соответствует **mycommand**.

Аналогичную задачу можно выполнить командой **kill mycommand**, которая также позволяет находить процессы по имени и отправлять им сигналы.

9. **Какая команда используется в `top`, чтобы убить процесс?**

В интерактивной утилите **top** для отправки сигнала процессу используется клавиша **k**.

После её нажатия программа предлагает указать **PID** процесса, а затем номер сигнала. По умолчанию обычно предлагается сигнал **15 (SIGTERM)**. При необходимости можно указать другой сигнал, например **9 (SIGKILL)** для принудительного завершения процесса.

10. **Как запустить команду с достаточно высоким приоритетом, не рискуя, что не хватит ресурсов для других процессов?**

Для запуска программы с изменённым приоритетом используется утилита **nice**. Если необходимо уменьшить влияние ресурсоёмкого процесса на остальные программы, ему назначают положительное значение **nice**.

Например: **nice -n 5 mycommand** или **nice -n 10 mycommand**.

Чем больше значение **nice**, тем ниже приоритет процесса при распределении процессорного времени. Поэтому запуск ресурсоёмкой команды с положительным значением **nice** позволяет другим процессам получать процессорные ресурсы с более высоким приоритетом.

3 Заключение

В ходе лабораторной работы были изучены основные средства управления заданиями и процессами в Linux. Были отработаны запуск процессов в фоновом и переднем режимах, приостановка процессов сочетанием Ctrl+Z, продолжение выполнения с помощью bg, перенос заданий на передний план командой fg и их завершение.

С помощью команд jobs, ps и программы top были рассмотрены различные способы получения информации о работающих процессах. Команды kill и killall использовались для отправки сигналов отдельным процессам и одновременного завершения группы процессов.

Также была изучена работа сигнала SIGTSTP и утилиты nohup, позволяющей процессу продолжать выполнение независимо от состояния терминальной сессии. С помощью nice и renice были рассмотрены механизмы изменения приоритета процессов. Установлено, что увеличение значения nice уменьшает приоритет процесса, а уменьшение значения nice, включая использование отрицательных значений, увеличивает его приоритет.